A Proof of the Proof

How to check the statement ledger, and what checking it establishes

Abstract

This is a user's document. It says what the published proof object claims, how a reader can check that claim with their own hands, what the reader should see when they do, and exactly how much the result is worth. It deliberately does not describe how any of the checking machinery is built: none of it has to be trusted or understood for the checks below to mean what they say, and a check that only works if you believe the checker is not a check. Everything reported in Section 6 was re-run for this document.

1 What the artifact is

The object being published is a *statement ledger*. It has two halves.

A machine-checked library.
A body of formal mathematics, written in a proof assistant, in which every result is proved from the kernel's axioms alone — no unproved assumptions, no gaps left for later, no escape hatches. The proof assistant's kernel, not the author, decides whether that is true, and it decides it again every time the library is built.

A cryptographic ledger over that library.
One entry per declaration in the library, 692 in all. Each entry names a declaration, records a digest of its exact statement, and carries a short, self-contained cryptographic proof that binds the two together and binds the whole entry into a single aggregate. The entries are independent of one another: any one of them can be checked on its own, in isolation, by anybody, forever, without contacting the issuer and without the other 691.

The point of the pair is that the second half is small and cheap to check while the first half is large and expensive. A reader who cannot read the mathematics, or who does not wish to install a proof assistant, can still check the ledger in under a minute and learn something real from it. A reader who wants the full statement builds the library too. Section 4 is precise about what each of those two readers ends up knowing.

2 The claim, stated plainly

> *For each of 692 named results, the issuer committed — publicly, at a fixed time, and irrevocably — to one exact statement, and holds an opening of that commitment; the statement is bound into the proof so the proof cannot be moved to a different statement; and the library that proves those statements builds cleanly with nothing assumed.*

Three things are worth separating out of that sentence, because they are checked by different means and they fail in different ways.

1. **Integrity of each entry.** The entry is internally consistent and accepting. A reader checks this with the verifier (Check 1). No library needed.

2. **Binding to a statement.** The entry is not a free-floating proof: it is tied to one specific statement text, and an entry lifted onto a different statement stops verifying. This is what makes the ledger a ledger *of statements* rather than a pile of unrelated proofs.

3. **Correspondence to real mathematics.** The statement the entry commits to is genuinely a theorem of a library that builds. A reader checks this by rebuilding the library and recomputing the digests (Checks 2 and 3).

3 How to use it

Four checks, in increasing order of cost. Each is independent; you may stop after any of them. The expected output is given for each, and each check either prints what is shown below or fails loudly — there is no third outcome, and no judgement call for the reader to make.

Check 1 — verify the ledger

one minute, no dependencies

The verifier is shipped in four independent implementations, so that agreement between them is itself evidence. Run whichever suits you; you need no package manager, no network and no library for any of them.

```
python3 qr/verifier/verifier.py      qr/verifier/ledger-bundle.txt
node    qr/verifier/verifier.js      qr/verifier/ledger-bundle.txt
node    qr/verifier/verifier-wasm.js qr/verifier/ledger-bundle.txt
rustc -O qr/verifier/verifier.rs -o /tmp/v && /tmp/v qr/verifier/ledger-bundle.txt
```

Each prints one line per entry and then

```
659/659 entries accepted
```

That is 658 individual entries plus the aggregate. Before trusting a run, make the verifier demonstrate that it can say *no*:

```
python3 qr/verifier/verifier.py --self-test
```

which constructs an honest entry and accepts it, then tampers with it and rebinds it to a different statement, and reports that both were rejected. A verifier that accepts everything would pass Check 1 vacuously; the self-test is what rules that out, and it is the reason it is there.

Check 2 — rebuild the library

one machine-hour

```
lake build
```

Completion with no errors *is* the check: the kernel has re-proved every result from scratch. Then confirm that nothing was assumed rather than proved, by searching the sources for the four constructs that would let an author smuggle in an unproved claim — a stated-but-unproved goal, an abandoned goal, a new axiom, and a native implementation substituted for a verified one:

```
rg -n 'sorry|admit|^axiom |@\[implemented_by' RequestProject/
```

Expected output: nothing at all. Finally, confirm the ledger's names are the library's names:

```
python3 verify/coverage.py
```

Expected: of the 692 committed names, **0** genuine declarations are missing. (Seven names are ones the proof assistant materialises only when something asks for them; the tool lists them separately.)

Check 3 — recompute the statement digests *minutes, after Check 2*

This is the step that ties the two halves together, and it is the one most worth doing, because it is the only one that can catch a ledger that is internally perfect but about nothing.

```
lake env lean verify/dump_statements.lean
python3 verify/statement_digests.py
```

The tool prints each declaration from your own build, digests it, and compares against the published digest. Expected: **393 of 692 exact matches**, of which **384** additionally close the entire chain *statement* → *digest* → *binding context* → *challenge* → *accepting proof* without reference to anything but your build and the published document. Disagreements are written out with both texts side by side, for the reader to judge. Section 5 explains why the number is 393 and not 692, and why that is a property of the publication rather than of the mathematics.

Check 4 — the paper path *optional*

The verifier, a reader for the printed codes, and a loader that reassembles them are all published as QR codes as well as source: 142 codes, each one labelled with what it is, which part of what it is, and the digest to check the result against. A reader with a camera and no working internet connection can photograph the sheets, rebuild the verifier from them, and run Check 1.

```
node qr/loader/loader.js qr/codes/verifier-python.part*.payload.txt > verifier.py
```

The loader refuses the result unless the reassembled bytes match the published digest, so a mis-scanned or substituted code cannot pass silently. This path exists so that the checkability of the ledger does not depend on any service, host or archive remaining alive.

4 What a passing run establishes — and what it does not

It is easy to over-read a wall of green. The following table is the honest reading. In it, *entry* means one of the 692 ledger records.

If you ran	you now know
Check 1 alone	Every entry is a well-formed, accepting proof, bound to one statement digest and to the ledger it sits in. Nobody can later swap the statement an entry refers to, and nobody without the issuer's secret could have produced the entry. You know nothing yet about what the statements *say*.
Checks 1–2	In addition: a library exists that builds with nothing assumed, and it contains a declaration for every name the ledger commits to.
Checks 1–3	In addition: for 393 entries, the declaration in your build is character for character the one the ledger commits to, so the committed statement is a theorem of a library you rebuilt yourself, and for 384 of them the chain from that text to an accepting proof closes end to end in your hands.

Two negatives are worth stating explicitly, because they are the two mistakes readers of such documents usually make.

- **The ledger does not make the mathematics true; the kernel does.** The cryptography establishes *who committed to what, and when, and that it has not changed since.* The truth of the statements rests entirely on Check 2. A ledger over a broken library would verify perfectly.
- **The library does not make the ledger honest; Check 3 does.** A library that builds and a ledger that verifies can, in principle, be about different things. Recomputing the digests is what excludes that, and it is why Check 3 is the substantive one.

5 Limits, defects and open points

Reported here rather than buried, because a verification document that lists no limits should not be believed.

1. **The printed appendix truncates 34 entries at page breaks.** When an entry crosses a page boundary, the remainder of the field it was in, and every field after it, are not typeset. Those 34 entries cannot be checked by a reader who has only the PDF; they are checkable from the machine-readable ledger. This is a typesetting defect and carries no cryptographic weight, but it does mean the paper document alone is not self-sufficient. *Fix: allow the transcript block to break across pages.*
2. **The aggregate identity does not reproduce.** The aggregate entry itself verifies. The published relationship between the aggregate and the individual entries, however, cannot be confirmed from the document, and the evidence says it is not merely unconfirmable but inconsistent: one of the entries needed to form it is among the truncated ones, and no completion of the truncated value that is consistent with what *is* printed satisfies the identity. Agreement by chance is excluded at overwhelming odds. No individual entry is affected. *This should be re-checked on the issuer's side.*

3. **Digest reproduction is toolchain-sensitive.** The 393 of Check 3 is not a bound on the mathematics; it is a measure of how much of the publication is reproducible by a third party. The remaining entries divide into ones whose text agrees but whose digest does not, and ones whose text differs in ways that are visibly presentational — variable names, binder order, universe naming, automatically generated declarations, and six that the appendix truncated. No case was found in which the rebuilt library states something *weaker* than the ledger commits to. *Fix: publish the pretty-printer configuration and the toolchain hash alongside the ledger; that alone would remove essentially all of this ambiguity, and is the single highest-value change to the format.*

4. **Hash-function assumptions are assumptions.** The binding and collision properties proved about the scheme are conditional on the hash function used; nothing here proves anything about that hash function itself, and nothing can.

5. **Secrecy is not what is being checked.** The checks above concern integrity and binding. The confidentiality properties of the scheme are established separately, by machine-checked proof, and are not something a reader can confirm by running a verifier.

6 Certificate of this run

Every figure below was produced by re-running the corresponding check for this document, not copied from an earlier report. Check 1 was re-run in three of the four shipped implementations — the fourth needs a compiler that was not installed on the machine used — and all three agree entry for entry. Check 4 was re-run by rebuilding the verifier from its printed codes: the reassembled file is byte-identical to the shipped one and passes its own self-test. Of the 692 committed names, 685 are present in the rebuilt library and the other seven are names the proof assistant materialises only on demand, so no genuine declaration is missing.

Check	**Result**	**Status**
Library builds, no errors	complete build	pass
Unproved goals, abandoned goals, added axioms	none, in any source file	pass
Ledger entries accepted, per implementation	659 / 659, three times	pass
Verifier rejects a tampered and a rebound entry	both rejected	pass
Committed names missing from the library	0 of 692	pass
Statement digests reproduced exactly from the build	393 of 692	partial
Full chain closed end to end from the build	384 of 692	partial
Verifier recovered from the printed codes	byte-identical, accepts	pass
Aggregate entry accepts	yes	pass
Aggregate equals the combination of the entries	does not reproduce	**fail**
Entries fully legible in the printed appendix	658 of 692	partial

The three rows that are not a clean pass are exactly the three defects of Section 5: two of them are defects of *the document* rather than of the object it describes, and the third is a discrepancy in the aggregate that leaves every individual entry intact. Nothing found in this run casts doubt on any individual entry, and nothing found in this run casts doubt on the library.

7 What you still have to trust

A short list, because it should be short, and because a reader is entitled to see it in one place.

1. The kernel of the proof assistant, and its standard library. This is the usual and unavoidable base of any formal development.

2. The hash function, and the hardness assumption the scheme rests on. Standard, explicit, and shared with essentially all deployed cryptography.

3. Nothing else. In particular you need not trust the issuer, this document, the verifier's authors, or any host or service: the verifier is small enough to read in an afternoon, comes in four independent implementations that must agree, can be recovered from printed codes with no network, and demonstrates on demand that it is capable of rejecting.

This document describes how to use the artifact and how to read the result. It does not document the construction of the library, of the scheme, or of the checking tools, and none of that is needed to perform any check in Section 3. Commands are given relative to the repository root.